

From raw race results

to a FAIR, quality-assured knowledge graph

Formula 1 World Championship results, 1950–2025. Data Management, Winter 2026/27.

Chirag Adhikari
Student A: FAIR assessment and metadata design

Manohar Zalki
Student B: Data quality and profiling

Nicolas Bonilla
Student C: Data cleaning and pipeline

76 seasons of F1 results with real quality issues

Every World Championship Grand Prix from Silverstone 1950 to Abu Dhabi 2025, from the Ergast database as served by the Jolpica-F1 API.

25,907
race results, one row per car per race

1,149
championship Grands Prix

817
drivers and 203 constructor entries

77
circuits in 34 countries

Relational

One fact table and four reference tables: joins and keys matter.

Real defects

76 seasons of rule changes and file exports, not planted errors.

A natural network

Drivers, teams, circuits and countries connect into a graph.

The curated package covers Grands Prix; sprint races are out of its scope.

CRISP-DM + Medallion architecture & automated execution

Three zones that only read the zone before them, and every CRISP-DM phase backed by evidence.

Bronze Raw zone

25,907

rows extracted, values unchanged, lineage to file and line

→

Silver Clean zone

55/55

validation gates before 25,873 clean rows are written

→

Gold Served zone

30,141

graph edges, plus the FAIR record, the dashboard and the assistant

1 Business understanding
research questions, FAIR goals

2 Data understanding
profiling: 29 issues with file and line

3 Data preparation
55 gates, an audit record per step

4 Modelling
property graph, podium-probability model

5 Evaluation
28 tests, assistant 25/25 benchmark

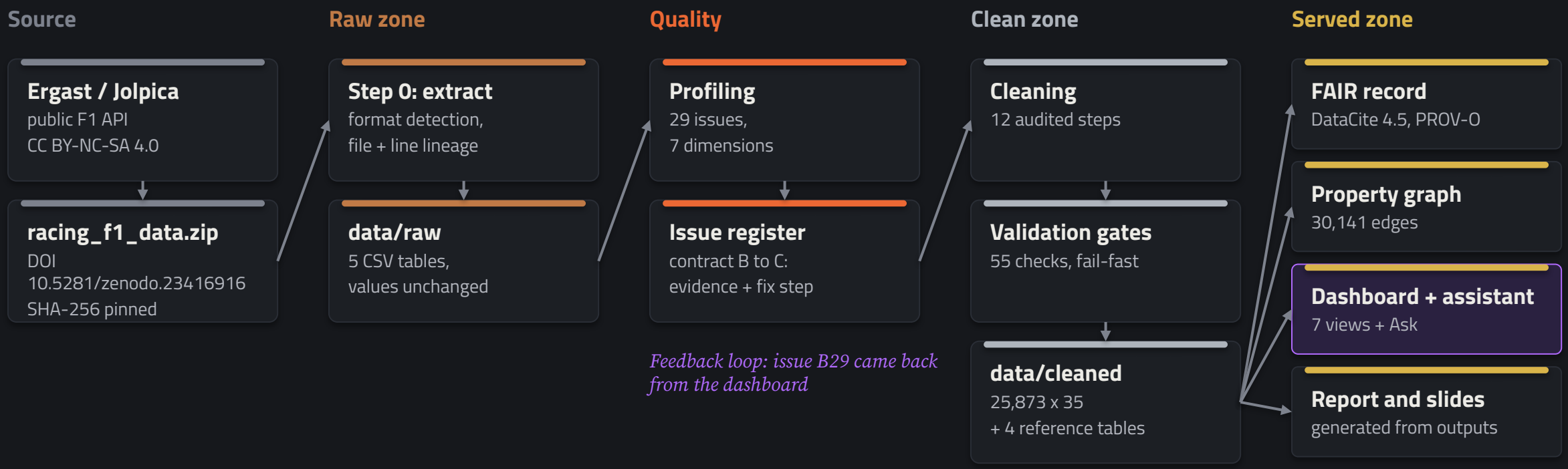
6 Deployment
dashboard and grounded assistant

```
python run_all.py --from-scratch
```

byte-identical reruns, 28 automated tests, no network needed

Data engineering: zones, contracts and controls

The issue register is the contract between profiling and cleaning; the schema module is the contract between cleaning and metadata.



Schema as code
one contract for C and A

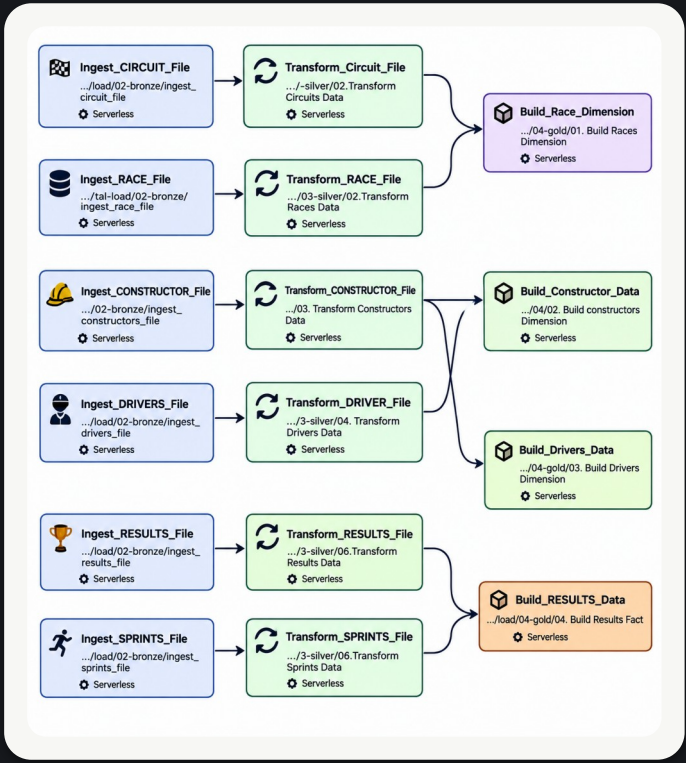
Orchestration
run_all.py, byte-identical reruns

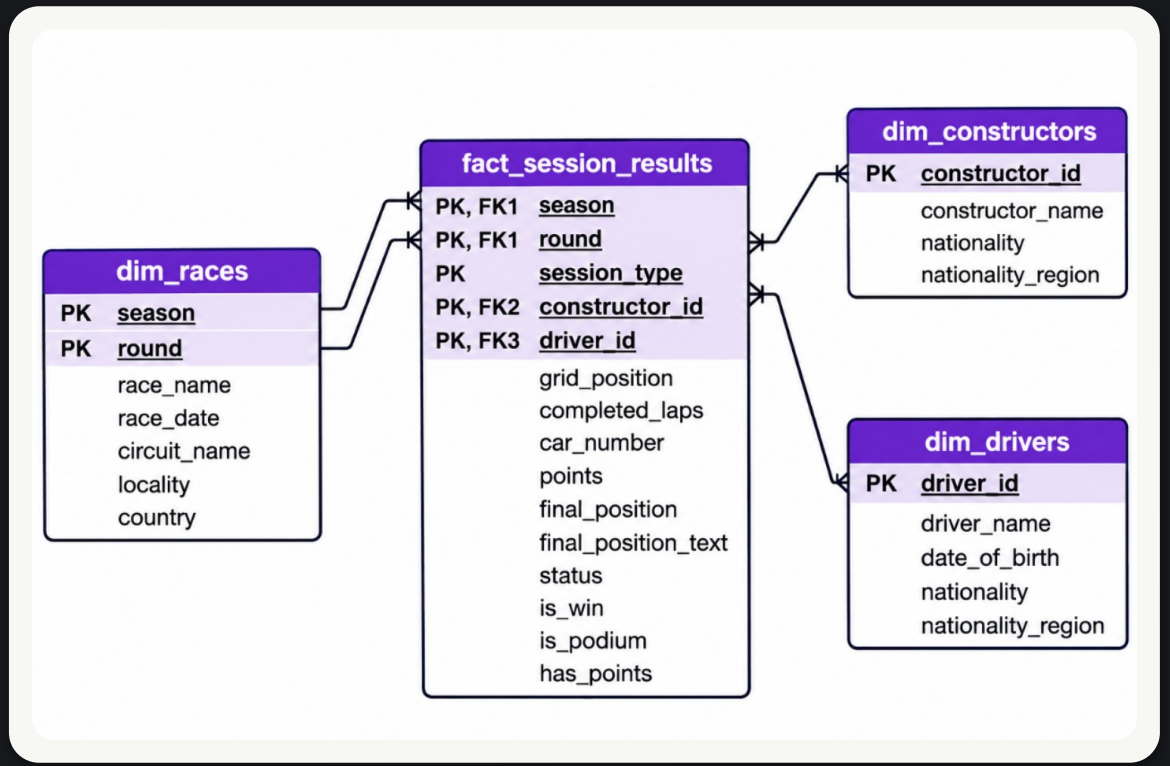
Automated tests
28 tests on key decisions

Lineage
file + line, checksums, PROV-O

The Spark medallion pipeline and its gold star schema

The incremental batch pipeline in PySpark notebooks: ingest each landing file to bronze, conform it in silver, model it as a star schema in gold.

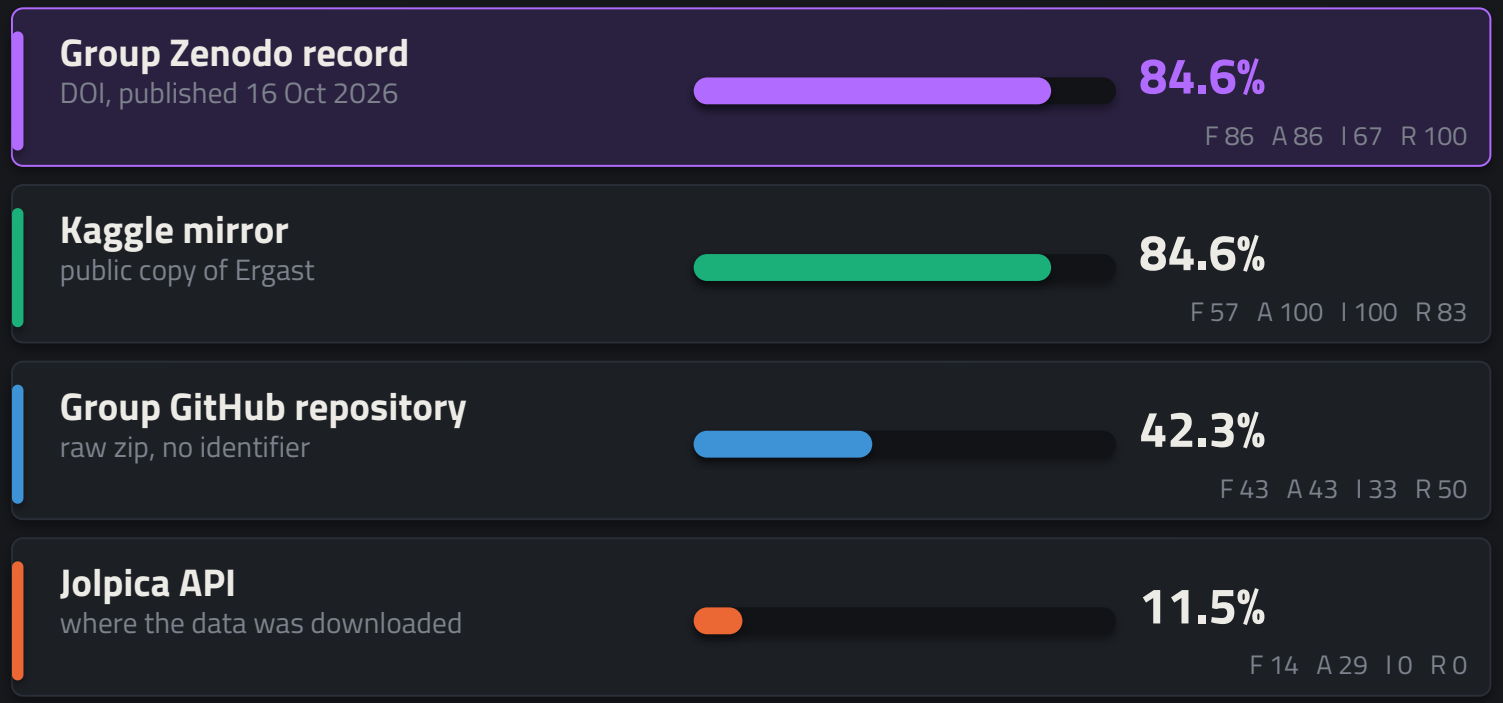




Gold also holds sprint sessions (session_type) and feeds the classic dashboard (RQ1-RQ6) and the ML prototype; the curated FAIR package covers Grands Prix only.

Same archive, two homes: 42.3% on GitHub, 84.6% on Zenodo

F-UJI 4.0.1 with metrics v0.8 (as on f-uji.net) on the four locations of the data; the Zenodo record has a DOI.



54% → 93%
RDA maturity over 41 indicators: as obtained → our package

6/20 → 12/20
essential indicators met

Findable stays at RDA level 0 until the curated package has its own DOI; the raw archive already has one.

F-UJI checks machine-readable signals only: it scores a licence as present, not as compatible. The RDA model also weighs licences and provenance.

Eliminating licensing ambiguity and metadata gaps

Five places publish the same results under four different terms. The data must stay CC BY-NC-SA 4.0.

Ergast (original source)	non-commercial use only	Origin
Jolpica-F1 (our download)	CC BY-NC-SA 4.0	Binding
Kaggle mirror	claims CC0, public domain	Conflict
Group GitHub repository	MIT licence for everything	Conflict
Group Zenodo record	CC BY 4.0	Correct it

Resolution data CC BY-NC-SA 4.0 with attribution; code MIT
Zenodo licence to be corrected to CC BY-NC-SA 4.0: a metadata edit, the DOI does not change

Persistent identifiers

10.5281/zenodo.23416916

DOI of the raw archive, byte-identical to our source (MD5 checked), cited as IsDerivedFrom

98.7% drivers linked to Wikidata (constructors 98.1%, circuits 100%)

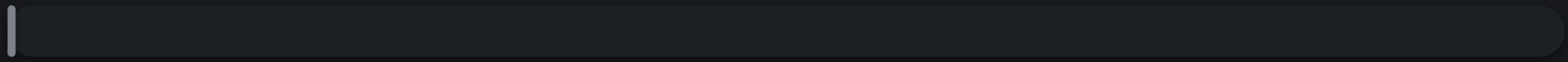
4.5 DataCite record validated against the official XSD

Next own DOI for the curated package, ORCID iDs, ROR

Uncovering disguised nulls behind surface completeness

The results table looks 99.9% complete. It is not: most of its gaps wear the costume of data.

Explicit nulls



18

Disguised missing values



9,913

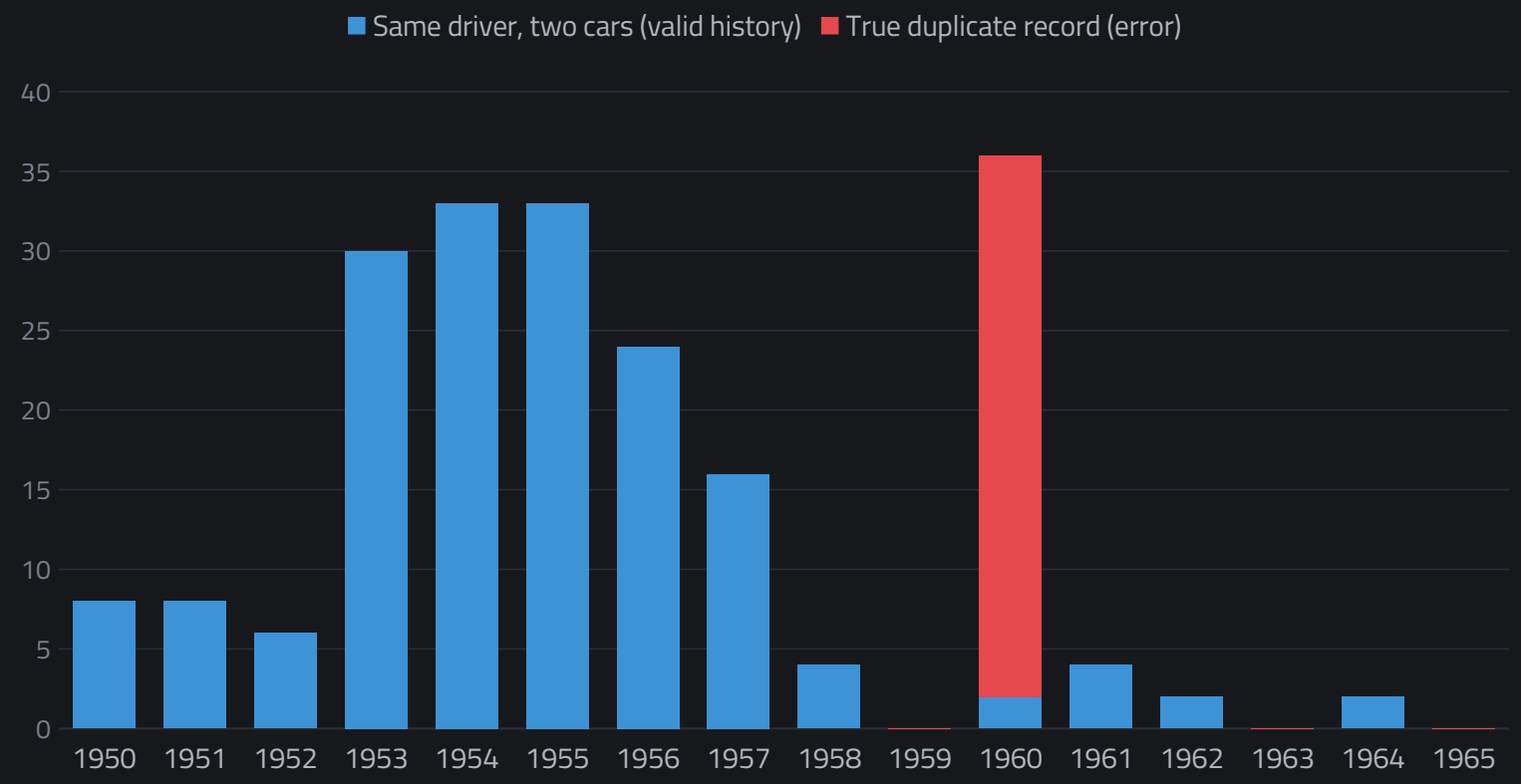
9,646
'positions' that are really retirements, withdrawals or disqualifications

267
cars on a fictitious 'grid slot 0'

18th
is how a model would read a car that retired

A naive duplicate check deletes real history

Rows sharing season, round and driver: true duplicates in red, legitimate shared drives in blue.



34
true duplicates, all from 1960

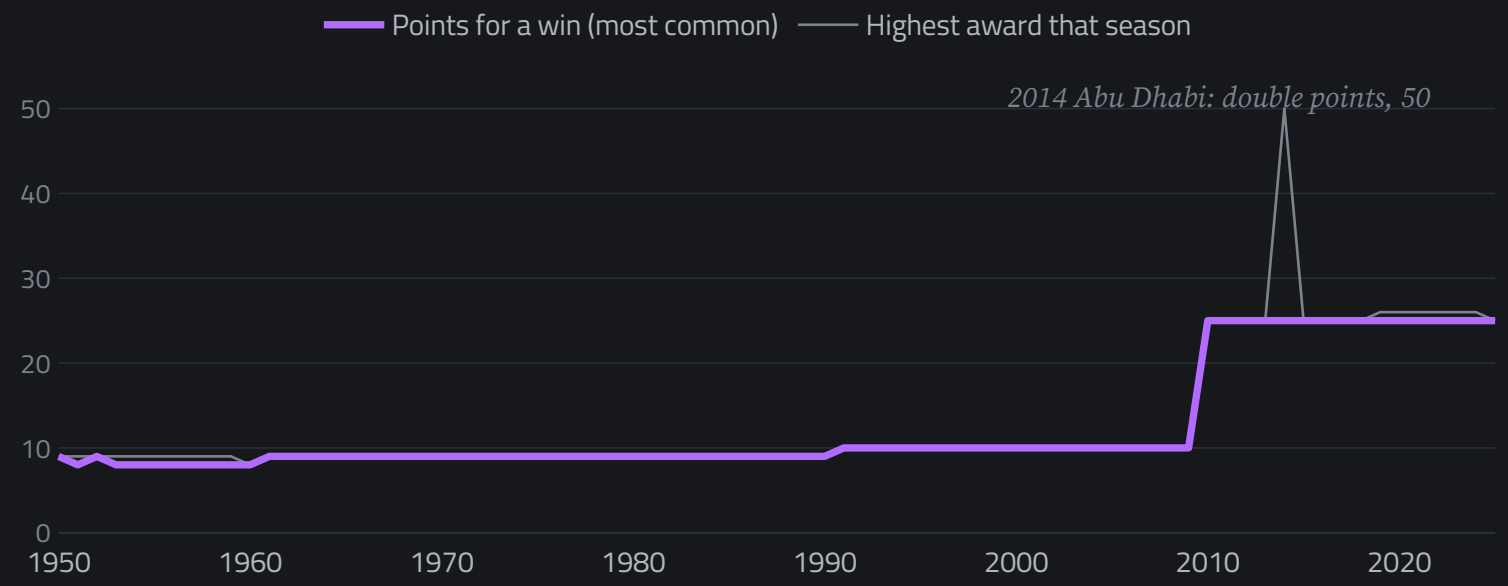
11
of them hidden behind blanked keys

172
valid rows: one driver, two cars in one race

Correct key season, round, driver and car number

Valid history can look like bad data

Points for winning a race, season by season: the same achievement, a different currency in every era.



8–50
points for the same race win

270
lap 'outliers', all valid Indianapolis 500 results: flag them, never delete

Impact on data science and AI

Inflated win counts
from duplicated rows

Biased targets
a retirement read as 18th

Era bias
raw points favour recent decades

Order matters: repair keys, then de-duplicate

The same two steps in the opposite order leave a third of the duplicates in the data.

Naive order

1 Remove exact duplicates
2 Repair missing keys

11 duplicates survive, hidden by the blanked keys

Our order: steps C02 → C03

1 Restore season (5) and round (7) by exact lookup
2 Remove exact duplicates

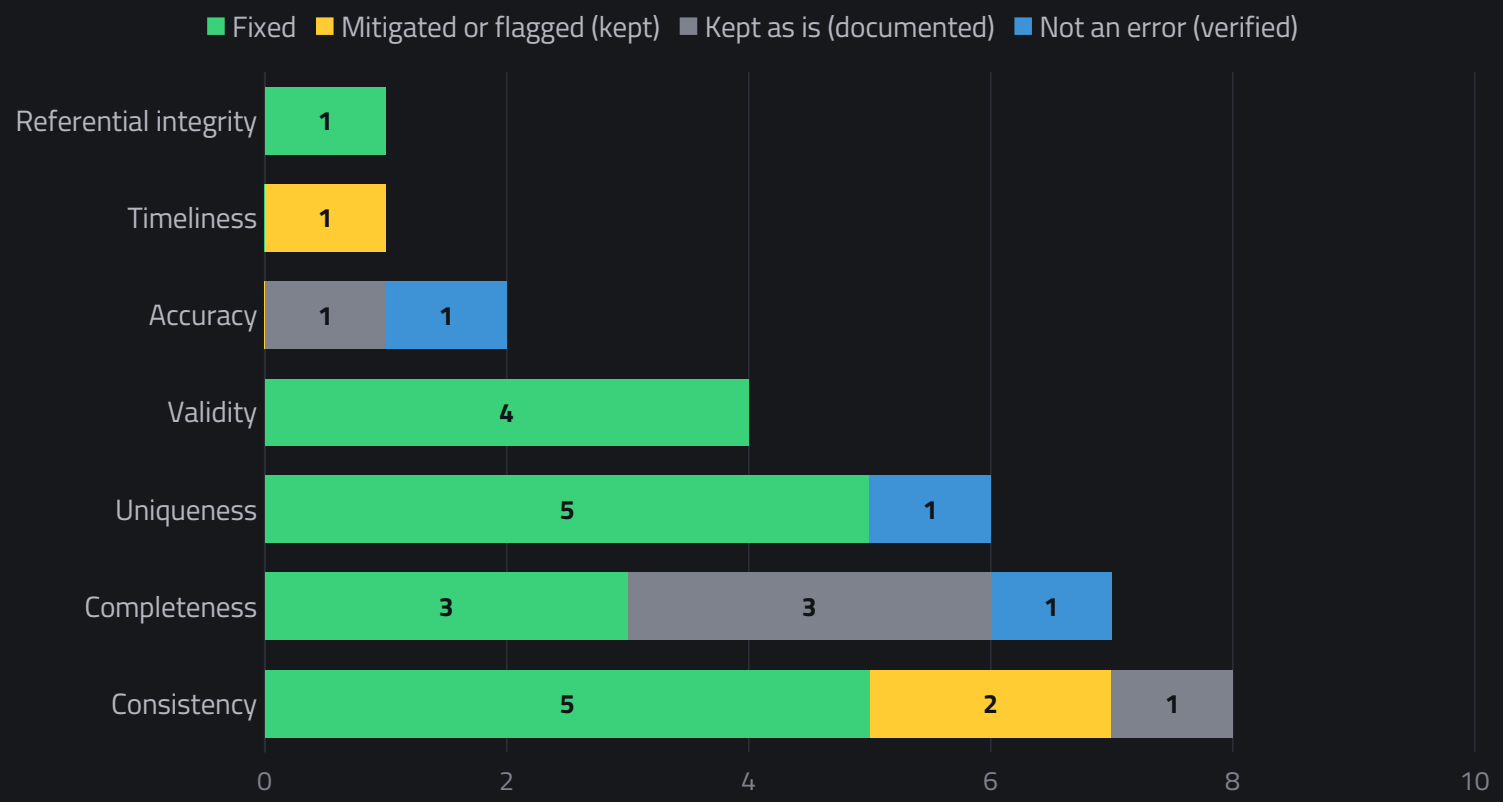
34 duplicates removed, none left; a unit test locks it in

No invented values

Grid 0 becomes missing plus a flag (267 rows); no statistical imputation anywhere.
Keys come only from deterministic lookups: the file name and races.csv.

Every issue closed, every output validated

How the 29 quality issues were resolved, per dimension. Flag, don't delete.



55/55
validation gates passed; one failure blocks publishing

18 of 29
issues fixed; the rest flagged, kept or verified

25,873
clean rows, 35 documented columns

75 years of F1 form a small world

A NetworkX property graph of 2,297 nodes and 30,141 typed edges; drivers linked when they shared a team in a Grand Prix.

0.41
clustering, against 0.014 in a random graph

4.8
team-mate links between two drivers on average

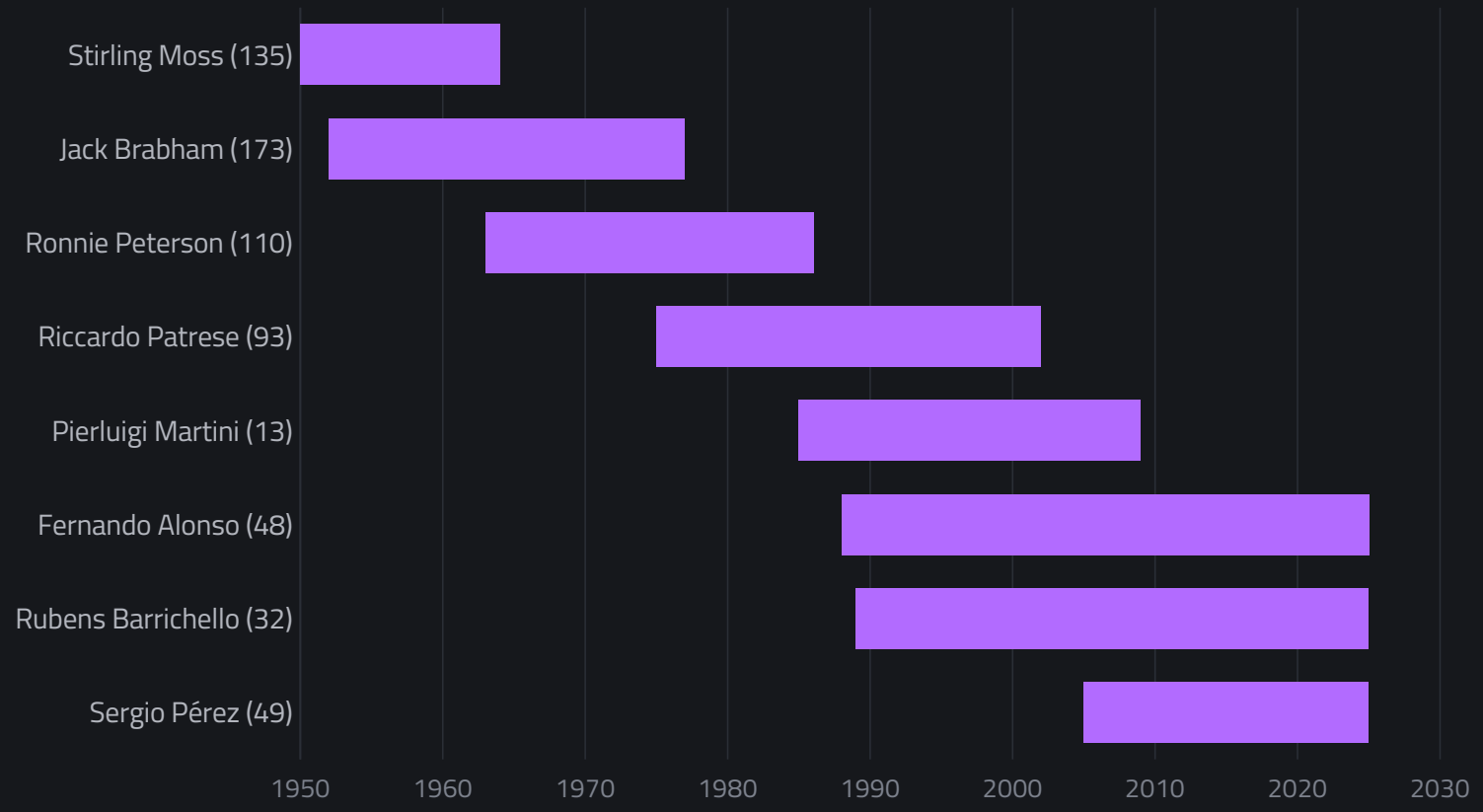
Fangio to Verstappen in 8 steps

Fangio → Bonnier → Andretti → Giacomelli → de Cesaris → Barrichello → Button → Pérez → Verstappen



Communities rediscover the eras by themselves

Louvain communities of the team-mate network: the algorithm never sees a date, yet each community is an era.



11

Louvain communities, modularity 0.70

0.15

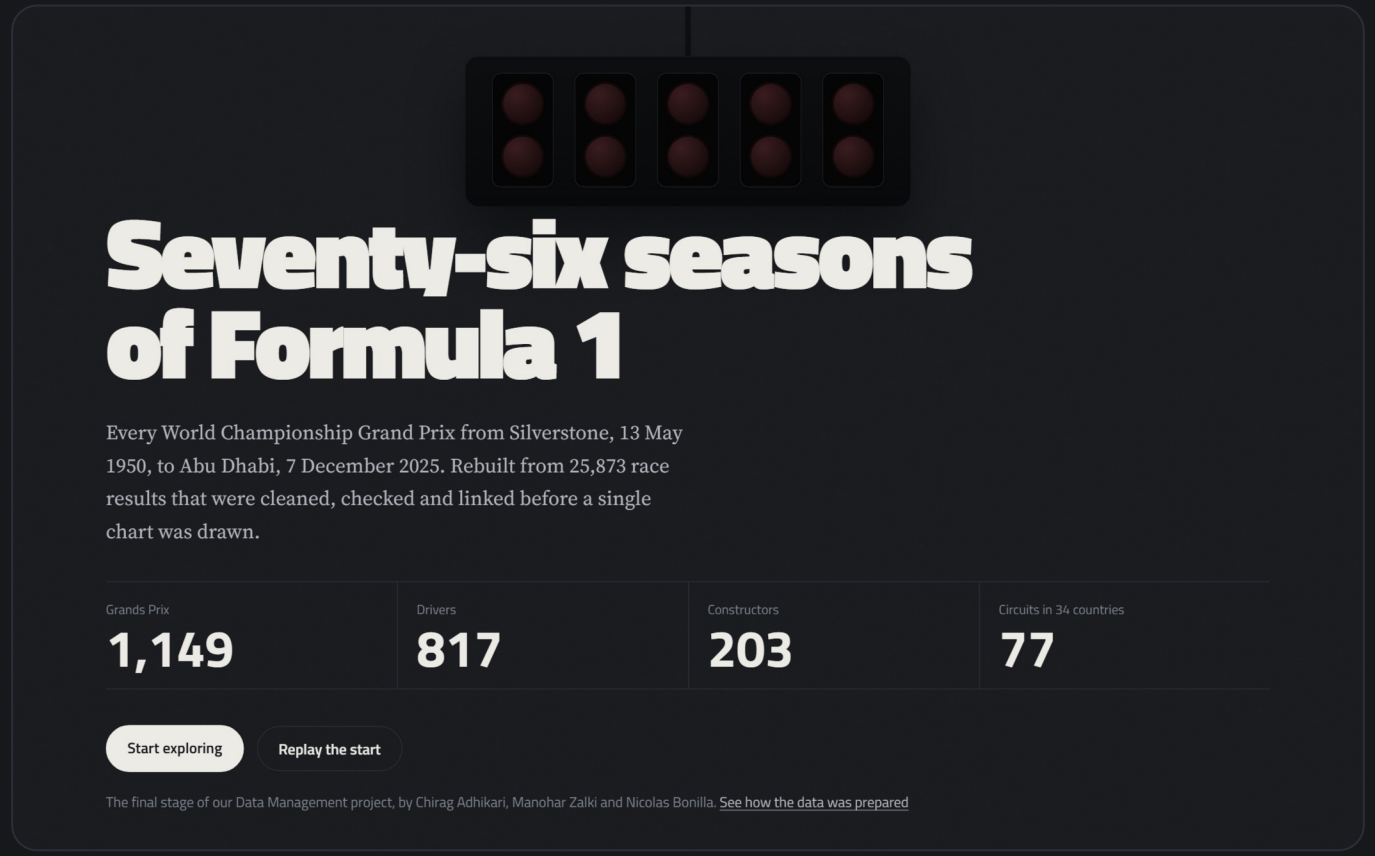
highest betweenness; bridges between eras: Bonnier, Brabham, de Cesaris

Caveat

Before the 1970s 'constructor' was the chassis make shared by private entrants, which inflates early degrees.

The project, live: our interactive dashboard

Seven views and an assistant, built only from validated outputs and rebuilt by run_all.py.



Validated data only
build_web_data.py reads the clean tables, graph metrics, issue register and FAIR scores

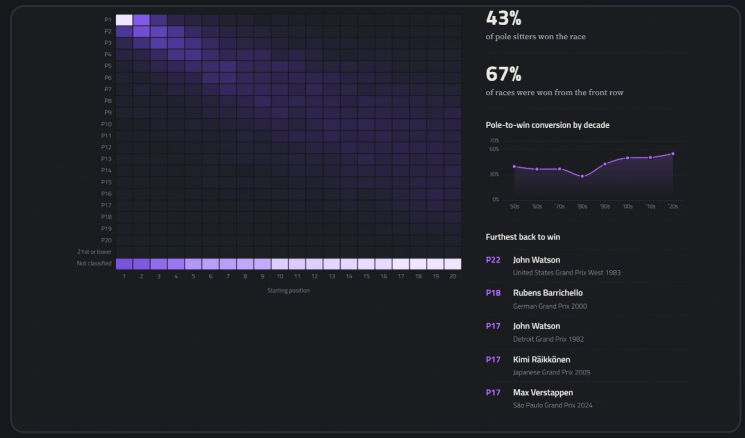
A quality instrument
its reliability view revealed issue B29: retirement causes not recorded from 2023

Live demo
`streamlit run app.py`
then localhost:8501

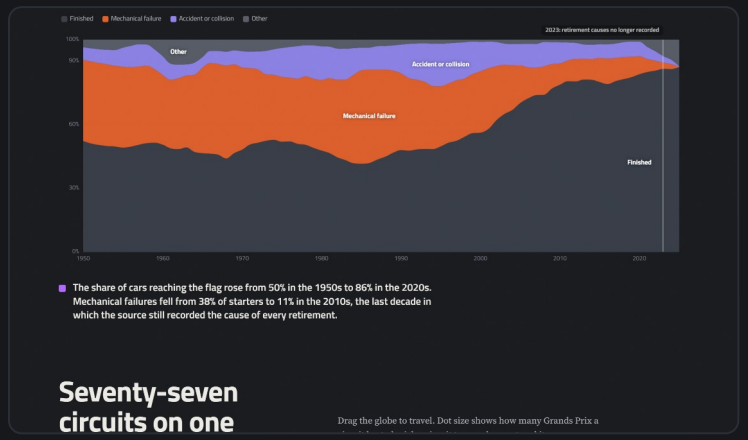
Inside the dashboard: every chart reads validated data



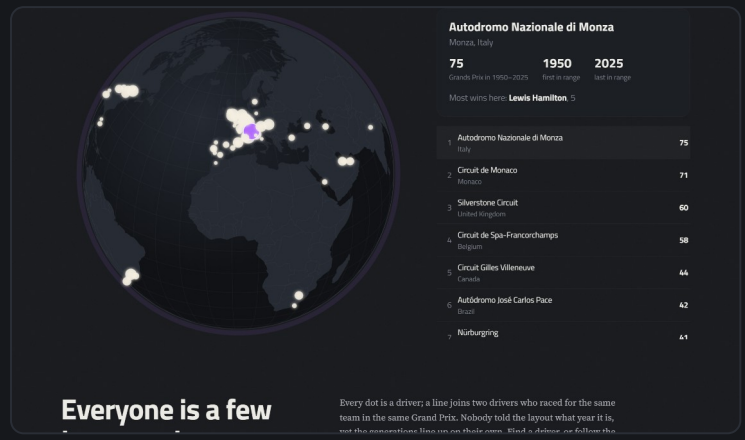
Timing tower: wins, podiums, era-adjusted score



Grid to finish, pole conversion, furthest-back wins



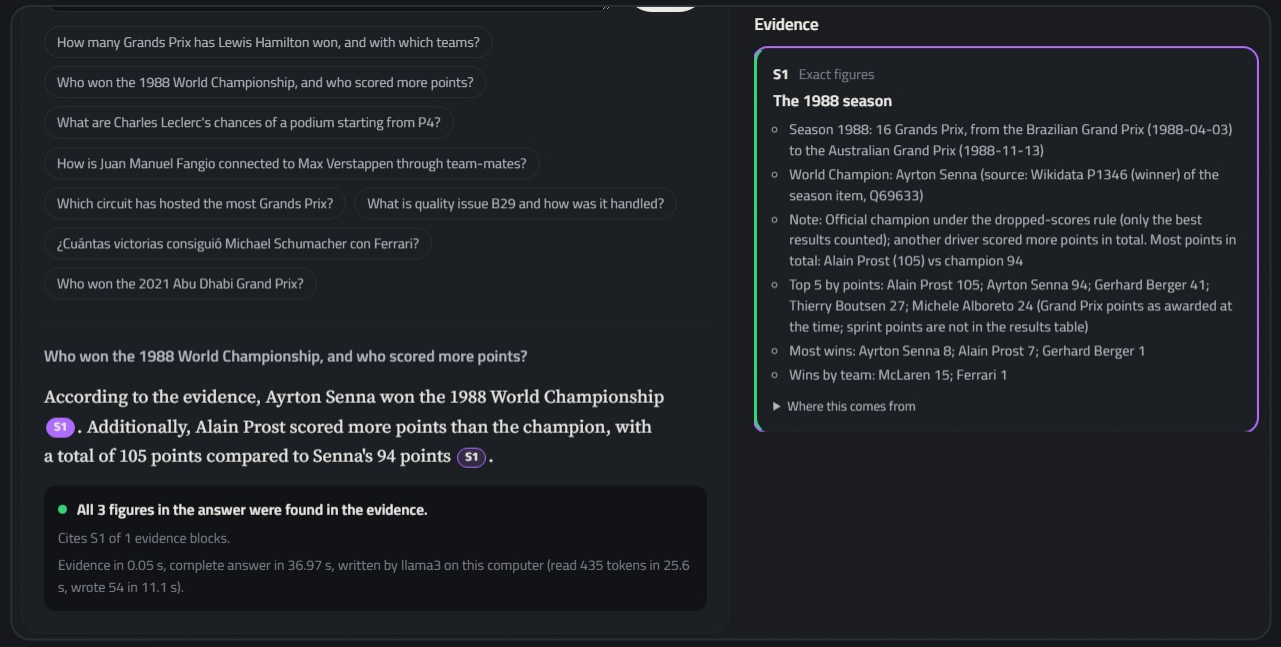
Reliability, with the 2023 break (B29) marked



77 circuits on a rotating globe

Ask the data: an LLM that cannot invent numbers

A local Llama 3 phrases the answer; the figures come from code over the validated data and are checked afterwards.



Understand
names, seasons, intent

Evidence
pandas, graph, model, documents

Write
Llama 3, cites [S1]

Check
every figure traced

25/25
benchmark answers correct, written by the model

49/49
figures in the answers traced to the evidence

26 s
median per answer on a laptop CPU, no GPU

Podium model, tested on 2020-2025
log loss 0.219 vs 0.254 for the grid alone, but no better podium picks: we say so

Idea and prototype: Manohar Zalki (Graphic-LlmRag)

What we learned

1 **Quality is contextual**
Outliers, duplicates and short grids were often valid history.

2 **Order of operations matters**
11 duplicates hid behind blanked keys.

3 **FAIR gaps sit around the values**
Identifiers, licences and provenance, not the numbers.

4 **Automate the evidence**
Audit log, 55 gates, 28 tests, metadata from code.

5 **A language model needs a data layer**
Code computes every figure; the model only phrases it, and a check follows.



Questions?

Lights out. Code, data, metadata, report and dashboard:

`github.com/zalkimanohar/Formula1-Racing-Analytics`

Chirag Adhikari **Manohar Zalki** **Nicolas Bonilla**